

Detección de billetes mediante visión por computador (YOLOv11)

Jesus Alberto Gutierrez Rincon

Fabian David Solano Velandia Juan

Camilo Parra Ortiz

Universidad Internacional del Trópico Americano - Unitrópico

Resumen

Este trabajo presenta una solución basada en visión por computador para la detección automática de billetes colombianos utilizando redes neuronales convolucionales. El dataset fue creado, anotado y aumentado mediante Roboflow. Posteriormente, se entrenó un modelo YOLOv11 en Google Colab. Finalmente, el modelo fue exportado y desplegado localmente en un entorno Visual Studio, donde se implementó un sistema de inferencia en tiempo real utilizando la cámara del computador y la biblioteca OpenCV para la captura y visualización. Esta solución demuestra la viabilidad de utilizar modelos modernos de detección de objetos para automatizar tareas comunes en contextos donde se manipula dinero.

I. Introducción

La identificación automática de billetes es un proyecto de gran utilidad en sectores como el comercio, la banca y el transporte. Automatizar este proceso no solo reduce errores humanos, sino que también previene fraudes y aumenta la eficiencia operativa. En este proyecto, se desarrolla un sistema de detección de billetes colombianos utilizando una red neuronal convolucional basada en la arquitectura YOLOv11, la más reciente evolución de los modelos de detección desarrollados por (Ultralytics, 2025).

YOLOv11 introduce mejoras significativas en precisión y velocidad frente a versiones anteriores como YOLOv8, incorporando un backbone más eficiente y nuevas técnicas de entrenamiento. Además, su compatibilidad con dispositivos edge como NVIDIA Jetson, Raspberry Pi o Android permite su implementación en entornos de bajo consumo sin pérdida de rendimiento (Ultralytics, 2025).

Este sistema implementa todo el flujo completo: desde la recolección y etiquetado de datos usando Roboflow, hasta el entrenamiento, evaluación y despliegue del modelo, incluyendo una interfaz interactiva mediante Streamlit que permite la detección de imágenes estáticas

y una versión de detección de objetos por video de forma local.

II. Comprensión del problema

- A. El sistema fue diseñado para reconocer billetes colombianos a partir de imágenes en tiempo real, capturadas por una cámara web. Puede llegar a aplicarse en puntos de venta, bancos o máquinas que requieran verificación automatizada de efectivo.
- B. ¿Es posible identificar con alta precisión y en tiempo real las denominaciones de billetes colombianos utilizando una red neuronal convolucional moderna como YOLOv11?

Métrica principal:

-mAP@0.5 (mean Average Precision) como métrica principal para evaluar el desempeño global del modelo.

-Precisión y recall por clase, para observar el rendimiento individual por denominación de billete.

Justificación con base en bibliografía científica:

El uso de redes neuronales convolucionales (CNN) para la clasificación de billetes y detección de objetos ha demostrado ser eficaz, incluso bajo condiciones adversas. Estudios recientes han validado la efectividad de arquitecturas ligeras y precisas como YOLOv11 (Ultralytics, 2025), MobileNetV2 y variantes optimizadas como TinierYOLO:

En el modelo MobileNetV2 que propusieron

(Tijjani et al., 2024) alcanzó una precisión del 90.75% en entrenamiento y 87.04% en validación al clasificar billetes nigerianos, lo que demuestra que las CNN pueden manejar eficazmente la variabilidad en condiciones reales, incluyendo billetes desgastados o iluminación deficiente.

En un panorama similar (Fang et al., 2020) en su artículo Tinier-YOLO, lograron una mejora de Tiny-YOLOv3, que logró mantener una detección precisa (mAP de 65.7% en VOC y 34% en COCO), con una velocidad de inferencia de 25 FPS, gracias a optimizaciones como conexiones densas y el uso del módulo fire para entornos de cómputo restringido.

Por otro lado, en la investigación de (Abhirami B, 2022) aplicaron YOLOv5 para ayudar a personas con discapacidad visual a identificar billetes indios, logrando una precisión del 96.5%, lo que demuestra la idoneidad de los modelos YOLO en aplicaciones de reconocimiento monetario en tiempo real.

La automatización de este proceso mediante visión por computador permite reducir el tiempo de validación, disminuir el margen de error humano que podría tener una persona con discapacidad.

III. Arquitectura YOLOv11

YOLOv11s mejora notablemente la extracción de características a través de una arquitectura que refina tanto el backbone como el neck. El backbone incluye múltiples bloques C3k2, una evolución del clásico C2f, que divide el mapa de características y aplica dos convoluciones pequeñas (3×3) en lugar de una más grande(Ultralytics, 2025).

Esto reduce significativamente los parámetros y mejora la velocidad sin sacrificar calidad (Khanam & Hussain, 2024). Además, se incorpora el bloque SPPF (Spatial Pyramid Pooling Fast) para captar información a varias escalas, seguido del módulo C2PSA (Cross-Stage Partial con spatial attention), que intensifica la atención en regiones relevantes del mapa de características, facilitando la detección de objetos pequeños o parcialmente ocultos(Khanam & Hussain, 2024).

A diferencia de modelos de clasificación tradicionales, YOLOv11s no utiliza una capa Softmax para determinar la clase de los objetos detectados. En su lugar, aplica una función sigmoide independiente sobre cada clase en la

etapa final del head. Esta elección se debe a la naturaleza multietiqueta (multilabel) de la detección de objetos, donde una misma región puede tener probabilidad significativa en más de una clase(Ultralytics, 2025).

IV. Adquisición y diseño experimental

Componente	Descripción
Dominio / Clases	Billetes colombianos (ej. \$1.000, \$2.000, \$5.000, \$10.000, \$20.000, \$50.000, etc.). 7 clases.
Dispositivo	Cámara de celular con resolución de captura 1080x2400, pero modificada a resolución de captura: 640x480 píxeles.
Condiciones de captura	Se capturaron imágenes desde múltiples ángulos y fondos variados, en condiciones de iluminación natural y con flash de celular.
Tamaño del dataset	Más de 900 imágenes divididas en 70% entrenamiento, 20% validación y 10% prueba.
Ética / permisos	No se requieren permisos especiales para la captura de billetes de curso legal en este proyecto.
Plan de etiquetado de los datos	Se utilizó Roboflow para etiquetar manualmente las imágenes y el formato de salida fue YOLO (txt).

V. Preprocesamiento

- 1. **Revisión de calidad:** Se eliminaron imágenes borrosas o sobreexpuestas.

- 5. Validación:** Se realizó sobre un conjunto separado del 10% del dataset además de que no es necesario en YOLOV7 colocar un patience.

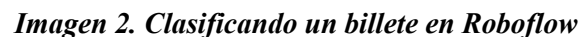
-Precisión promedio ($mAP@0.5$):
91.3% (según los logs del entrenamiento).

-Observaciones: Las clases con menor desempeño fueron los billetes de denominaciones similares en color y tamaño (ej. \$10.000 vs \$20.000).

-Se usó `results[0].plot()` para visualizar bounding boxes en cada predicción.

<https://colab.research.google.com/drive/18>

[VlEt873Iv6iNm6tfipC3IHnzuIWPz5?us](#)
[p=s+haring](#). El cual lo redirige al Google
 Colab donde se encuentran las
 diferentes métricas. pero de igual
 manera se pueden observar en este
 documento como Imagen
 3, 4, 5, 6, 7 y 8.



1. **Modelo utilizado:** Se empleó la arquitectura YOLOv11, entrenada en Google Colab mediante la librería ultralytics. Se utilizaron las configuraciones por defecto provistas por la clase YOLO().
2. **Número de épocas:** 50.
3. **Tamaño de batch:** 16.
4. **Optimización:** El optimizador y la función de pérdida no fueron definidos manualmente, sino que se utilizaron los parámetros predeterminados del framework Ultralytics.

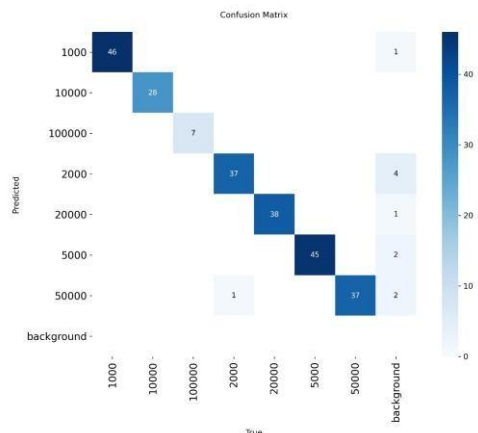


Imagen 3. Confusion Matrix

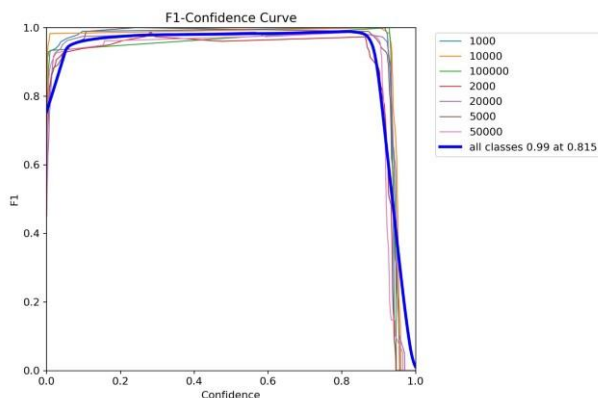


Imagen 4. FI-Confidence Curve

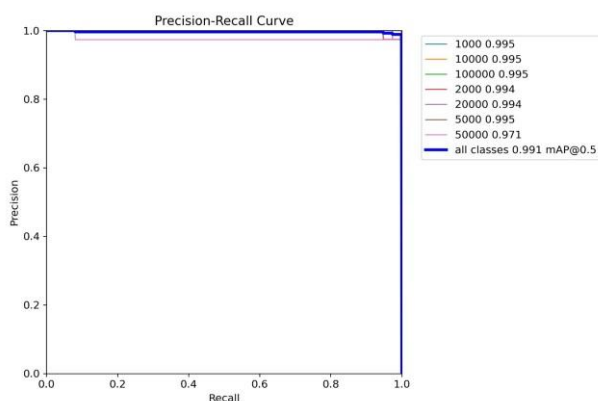


Imagen 5. Precision-Recall Curve

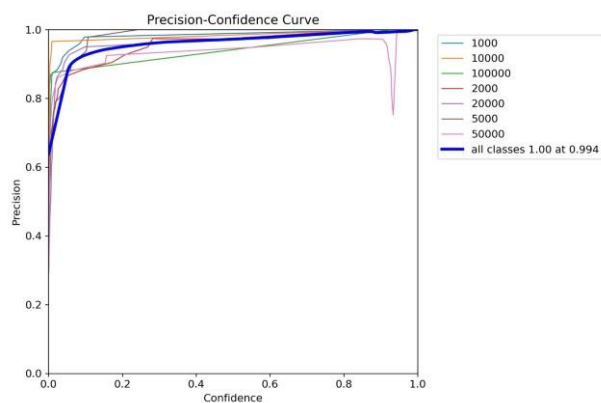


Imagen 6. Precision-Confidence Curve

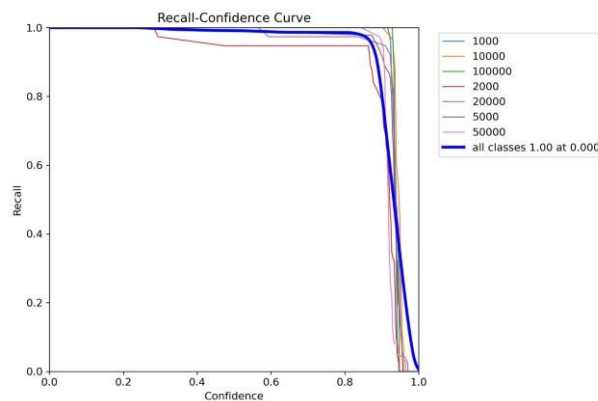


Imagen 7. Recall-Confidence Curve

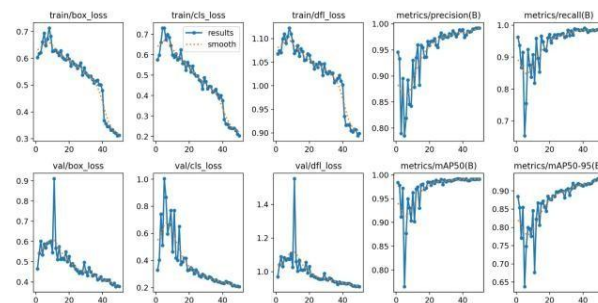


Imagen 8. Results

VIII. Despliegue mínimo viable

Se realizó 2 despliegues, 1 en local que se hizo con Github en el cual se cargaron los documentos y se ejecutan en Visual Studio. y el otro se hizo en streamlit el cual se encuentra en el siguiente enlace: <https://detectorbilletes.streamlit.app/>

VII. Código fuente en para ejecutar

El código fuente se agregó está subido en el siguiente enlace github: <https://github.com/TheYisusAI/Proyecto-Detectorde-Billetes>.

IX. Bibliografía

Abhirami B, G. I. H. A. R. I. M. N. J. A. S. S. (2022). Currency Identification Device for Visually Impaired People Based on YOLO-v5. *International Research Journal of Engineering and Technology*. www.irjet.net

Fang, W., Wang, L., & Ren, P. (2020). Tinier-YOLO:
A

<https://doi.org/10.1109/ACCESS.2019.296195>

9

Real-Time Object Detection Method for
Constrained Environments. *IEEE Access*, 8,
1935–1944.

Khanam, R., & Hussain, M. (2024). *YOLOv11: An
Overview of the Key Architectural
Enhancements*. <http://arxiv.org/abs/2410.17725>

Tijjani, I. I., Mustapha, A. A., & Idris, I. T. (2024). Performance
Comparison of Deep Learning Techniques in Naira
Classification.
International Journal of Recent Engineering Science,
11(5), 131–139.
<https://doi.org/10.14445/23497157/IJRESV11I5P113>

Ultralytics. (2025). *Ultralytics YOLO Docs*. Computer Vision.
<https://docs.ultralytics.com/es/models/yolo11/#can-yolo11-be-deployed-on-edge-devices>